# Phase 2 - Model Checking using SPIN

After automatically generating the Promela model in Phase 1, the generated model was verified using the SPIN model checker. To perform formal verification, several Linear Temporal Logic (LTL) properties were automatically appended to the generated Promela file. Each property was then verified independently using SPIN.

Four verification properties are generated for the translated model.

- Safety
- Liveness
- Reachability
- Invariant

The translator automatically generates these properties through four helper methods.

- **ltlSafety()** generates a safety property ensuring that no division-by-zero occurs during execution.
- **ltlLiveness()** generates one LTL property for every loop in the program, ensuring that entering a loop eventually leads to exiting it.
- **ltlReachability()** checks whether the program can eventually reach its final state.
- **ltlInvariant()** generates an invariant property that verifies a specified condition remains true during the entire execution.

After generating the Promela model, these LTL specifications are appended to the end of the generated file and verified using SPIN.

# Verification Examples

## Example 1 - Successful Verification

### Hash Source

```
adad x = 10;
adad y = 2;
adad z = x / y;
```

### Generated Promela

```
/* Generated Promela model from Hash */
bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;
int y;
int z;

active proctype main() {
    x = 10;
    y = 2;
    if
    :: (y == 0) ->
    divByZero = true;
    goto endReached_label;
    :: else ->
    skip;
    fi;
    z = x/y;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/
ltl safety {
    [] (!divByZero)
}

ltl reachability {
    <>endReached
}

ltl invariant {
    [](x >= 0)
}
```

### Safety Verification

### Command

```
spin -search -ltl safety output.pml
```

### Output

```
pan: ltl formula safety

(Spin Version 6.5.2 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim  + (safety)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 17, errors: 0
9 states, stored
1 states, matched
10 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.289 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage


unreached in proctype main
output.pml:17, state 4, "divByZero = 1"
(1 of 13 states)
unreached in claim safety
_spin_nvr.tmp:8, state 10, "-end-"
(1 of 10 states)

pan: elapsed time 0 seconds
```

### Explanation

The divisor is never equal to zero. Therefore, the variable `divByZero` is never set to true and the Safety property is successfully satisfied.

### Reachability Verification

### Command

```
spin -search -ltl reachability output.pml
```

### Output

```
pan: ltl formula reachability

(Spin Version 6.5.2 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim  + (reachability)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 12, errors: 0
7 states, stored (14 visited)
5 states, matched
19 transitions (= visited+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.289 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage


unreached in proctype main
output.pml:17, state 4, "divByZero = 1"
output.pml:27, state 13, "-end-"
(2 of 13 states)
unreached in claim reachability
_spin_nvr.tmp:15, state 6, "-end-"
(1 of 6 states)

pan: elapsed time 0 seconds
```

**Explanation**

The execution reaches the final state of the program. Therefore, the Reachability property is satisfied.

### Deadlock Verification

### Command

```
spin -search -deadlock output.pml
```

### Output

```
proctype main
state 1 -(tr 9)-> state 2 [id 0 tp 2] [----G] output.pml:13 => x = 10
state 2 -(tr 10)-> state 8 [id 1 tp 2] [----G] output.pml:14 => y = 2
state 8 -(tr 11)-> state 4 [id 2 tp 2] [----G] output.pml:16 => ((y==0))
state 8 -(tr 2)-> state 7 [id 5 tp 2] [----G] output.pml:16 => else
state 4 -(tr 12)-> state 12 [id 3 tp 2] [----G] output.pml:17 => divByZero = 1
state 12 -(tr 1)-> state 13 [id 11 tp 2] [--e-L] output.pml:26 => (1)
state 13 -(tr 15)-> state 0 [id 12 tp 3500] [--e-L] output.pml:27 => -end-
state 7 -(tr 1)-> state 10 [id 6 tp 2] [----L] output.pml:20 => (1)
state 10 -(tr 13)-> state 11 [id 9 tp 2] [----G] output.pml:22 => z = (x/y)
state 11 -(tr 14)-> state 12 [id 10 tp 2] [----G] output.pml:24 => endReached = 1
claim safety
state 6 -(tr 7)-> state 6 [id 13 tp 2] [----G] output.pml:3 => (!(!(divByZero)))
state 6 -(tr 1)-> state 6 [id 16 tp 2] [----G] output.pml:3 => (1)
claim reachability
state 3 -(tr 5)-> state 3 [id 23 tp 2] [-a--G] output.pml:13 => (!(endReached))
claim invariant
state 6 -(tr 3)-> state 6 [id 29 tp 2] [----G] output.pml:19 => (!((x>=0)))
state 6 -(tr 1)-> state 6 [id 32 tp 2] [----G] output.pml:19 => (1)

Transition Type: A=atomic; D=d_step; L=local; G=global
Source-State Labels: p=progress; e=end; a=accept;
Note: statement merging was used. Only the first
stmnt executed in each merge sequence is shown
(use spin -a -o3 to disable statement merging)

pan: elapsed time 4.3e+06 seconds
pan: rate 0 states/second
```

### Explanation

SPIN explores the generated state space and reports no deadlock or invalid end state. The generated model is therefore deadlock-free.

### Invariant Verification

### Command

```
spin -search -ltl invariant output.pml
```

### Output

```
pan: ltl formula invariant

(Spin Version 6.5.2 -- 6 December 2019)
+ Partial Order Reduction

Full statespace search for:
never claim  + (invariant)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 17, errors: 0
9 states, stored
1 states, matched
10 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.289 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage


unreached in proctype main
output.pml:17, state 4, "divByZero = 1"
(1 of 13 states)
unreached in claim invariant
_spin_nvr.tmp:24, state 10, "-end-"
(1 of 10 states)

pan: elapsed time 0 seconds
```

### Explanation

The invariant specifies that the value of `x` must remain non-negative throughout execution. Since `x` is initialized to 10 and is never modified, the invariant holds for every reachable state, and SPIN reports no property violation.

## Example 2 - Safety Property Violation

### Hash Source

```
adad x = 10;
adad y = 0;
adad z = x / y;
```

### Generated Promela

```
/* Generated Promela model from Hash */

bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;
int y;
int z;

active proctype main() {
    x = 10;
    y = 0;
    if
    :: (y == 0) ->
    divByZero = true;
    goto endReached_label;
    :: else ->
    skip;
    fi;
    z = x/y;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/

ltl safety {
    [] (!divByZero)
}

ltl reachability {
    <>endReached
}

ltl invariant {
    [](x >= 0)
}
```

### Verification Command

```
spin -search -ltl safety output.pml
```

### Verification Output

```
pan: ltl formula safety
pan:1: assertion violated !( !( !(divByZero))) (at depth 20)
pan: wrote output.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
+ Partial Order Reduction

Full statespace search for:
never claim  + (safety)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 20, errors: 1
11 states, stored
0 states, matched
11 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.001 equivalent memory usage for states (stored*(State-vector + overhead))
0.286 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage



pan: elapsed time 0 seconds
```

### Explanation

The divisor becomes zero before the division operation. The translator raises the `divByZero` flag, violating the Safety property.

## Example 3 - Liveness Property Violation

### Hash Source

```
adad x = 5;
ta (x > 0) {
    edame;
}
```

### Generated Promela

```
/* Generated Promela model from Hash */

bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;

active proctype main() {
    x = 5;
    loop_start_1:
    do
    :: (x>0) ->
    inLoop_1:
    goto loop_start_1;

    :: else -> break
    od;
    exitLoop_1:
    skip;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/

ltl safety {
    [] (!divByZero)
}

ltl liveness_1 {
    [](main@inLoop_1 -> <>main@exitLoop_1)
}

ltl reachability {
    <>endReached
}
```

```

ltl invariant {
    [](x >= 0)
}
```

### Verification Command

```
spin -search -ltl liveness_1 output.pml
```

### Verification Output

```
pan: ltl formula liveness_1
pan:1: acceptance cycle (at depth 6)
pan: wrote output.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
+ Partial Order Reduction

Full statespace search for:
never claim  + (liveness_1)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 9, errors: 1
5 states, stored
0 states, matched
5 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.287 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage




pan: elapsed time 0 seconds
```

### Explanation

The loop variable never changes. Therefore the loop cannot terminate and the exit label is never reached, which violates the Liveness property.

## Example 4 - Reachability Property Violation

### Hash Source

```
adad x;
ta (dorost) {}
```

### Generated Promela

```
/* Generated Promela model from Hash */
bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;

active proctype main() {
    loop_start_1:
    do
    :: (true) ->
    inLoop_1:
    skip;

    :: else -> break
    od;
    exitLoop_1:
    skip;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/

ltl safety {
    [] (!divByZero)
}

ltl liveness_1 {
    [](main@inLoop_1 -> <>main@exitLoop_1)
}

ltl reachability {
    <>endReached
}

ltl invariant {
    [](x >= 0)
}
```

### Verification Command

```
spin -search -ltl reachability output.pml
```

### Verification Output

```
pan: ltl formula reachability
pan:1: acceptance cycle (at depth 0)
pan: wrote output.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
+ Partial Order Reduction

Full statespace search for:
never claim  + (reachability)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 3, errors: 1
2 states, stored
0 states, matched
2 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.287 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage




pan: elapsed time 0 seconds
```

### Explanation

The loop condition is always true. The execution never reaches the final label, thus the Reachability property is violated.

## Example 5 - Deadlock Checking

### Hash Source

```
adad x = 0;
x = x + 1;
```

### Generated Promela

```
/* Generated Promela model from Hash */

bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;

active proctype main() {
    x = 0;
    x = x+1;
    ;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/

ltl safety {
    [] (!divByZero)
}

ltl reachability {
    <>endReached
}

ltl invariant {
    [](x >= 0)
}
```

### Verification Command

```
spin -search -d output.pml
```

### Verification Output

```
proctype main
state 1 -(tr 9)-> state 2 [id 0 tp 2] [----G] output.pml:11 => x = 0
state 2 -(tr 10)-> state 3 [id 1 tp 2] [----G] output.pml:12 => x = (x+1)
state 3 -(tr 11)-> state 4 [id 2 tp 2] [----G] output.pml:15 => endReached = 1
state 4 -(tr 1)-> state 5 [id 3 tp 2] [--e-L] output.pml:17 => (1)
state 5 -(tr 12)-> state 0 [id 4 tp 3500] [--e-L] output.pml:18 => -end-
claim safety
state 6 -(tr 7)-> state 6 [id 5 tp 2] [----G] output.pml:3 => (!(!(divByZero)))
state 6 -(tr 1)-> state 6 [id 8 tp 2] [----G] output.pml:3 => (1)
claim reachability
state 3 -(tr 5)-> state 3 [id 15 tp 2] [-a--G] output.pml:13 => (!(endReached))
claim invariant
state 6 -(tr 3)-> state 6 [id 21 tp 2] [----G] output.pml:19 => (!((x>=0)))
state 6 -(tr 1)-> state 6 [id 24 tp 2] [----G] output.pml:19 => (1)

Transition Type: A=atomic; D=d_step; L=local; G=global
Source-State Labels: p=progress; e=end; a=accept;
Note: statement merging was used. Only the first
stmnt executed in each merge sequence is shown
(use spin -a -o3 to disable statement merging)

pan: elapsed time 4.3e+06 seconds
pan: rate 0 states/second
```

**Explanation**

SPIN explores the entire state space and reports no deadlock or invalid end state. Therefore the generated model is deadlock free.

## Example 6 - Invariant Violation

### Hash Source

```
    adad x = 2;

    ta (x >= 0) {
        x = x - 1;
    }
```

### Generated Promela

```
/* Generated Promela model from Hash */

bool divByZero = false;
bool outOfBound = false;
bool nullPointer = false;
bool noPermission = false;
bool endReached = false;
int x;

active proctype main() {
    x = 2;
    loop_start_1:
    do
    :: (x>=0) ->
    inLoop_1:
    x = x-1;
    ;

    :: else -> break
    od;
    exitLoop_1:
    skip;

    endReached = true;
    endReached_label:
    skip;
}

/*
------------------------
ltl checker methods
------------------------
*/

ltl safety {
    [] (!divByZero)
}

ltl liveness_1 {
    [](main@inLoop_1 -> <>main@exitLoop_1)
}

ltl reachability {
```

```
    <>endReached
}

ltl invariant {
    [](x >= 0)
}
```

### Verification Command

```
spin -search -ltl invariant output.pml
```

### Verification Output

```
pan: ltl formula invariant
pan:1: assertion violated !( !((x>=0))) (at depth 14)
pan: wrote output.pml.trail

(Spin Version 6.5.2 -- 6 December 2019)
Warning: Search not completed
+ Partial Order Reduction

Full statespace search for:
never claim  + (invariant)
assertion violations + (if within scope of claim)
acceptance cycles  + (fairness disabled)
invalid end states - (disabled by never claim)

State-vector 28 byte, depth reached 14, errors: 1
8 states, stored
0 states, matched
8 transitions (= stored+matched)
0 atomic steps
hash conflicts: 0 (resolved)

Stats on memory usage (in Megabytes):
0.000 equivalent memory usage for states (stored*(State-vector + overhead))
0.287 actual memory usage for states
128.000 memory used for hash table (-w24)
0.534 memory used for DFS stack (-m10000)
128.730 total actual memory usage




pan: elapsed time 0.01 seconds
```

### Explanation

The invariant under verification is that the variable `x` must remain non-negative. During execution this condition is violated, causing SPIN to report an assertion violation.